

TCP协议栈开发设计

TCP协议栈开发设计

一、系统数据组织架构图

二、配置

1、VS配置组织架构图

2、POOL

a、REAL SERVER(RS)

b、POOL 基本结构

3、SNAT

a、基本结构

4、VS

a、基本结构

三、TCP & PROXY

1、TCP状态机

a、TCP状态

b、状态变换图

c、发送和接收窗口

d、状态机处理基本原则

e、状态机处理

f、序号判断

2、滑动窗口(数据包合并)

3、TCP的超时与重传

a、TCP传输的理想状态

b、慢启动和拥塞避免

i、Reno

ii、CUBIC

c、快速重传和快速回复

d、重启空闲连接

e、nagle算法

4、TCP选项

a、必须支持的选项：

- b、可选支持

5、定时器

- a、零窗口探测定时器(persist timer)

- b、重传定时器

- i、RTT 测量

- ii、重传定时器

- iii、RTO算法

- c、延迟ACK定时器

- d、TCB超时定时器

6、协议间联动

- a、ICMP / ARP(NDP) 与 TCP 的联动

- b、实现状态

四、其他

1、会话表

- a、基本结构

- b、会话表管理

- c、会话表创建

- d、会话表清理

- e、会话表查询

2、源端口搜索

- a、端口搜索

- b、端口回收

- c、删除

3、延迟队列

4、(缓存下一跳)

一、系统数据组织架构图

进程启动后首先会初始化一个 **ROOT 根对象**，其中保存了所有全局信息(有多少个NUMA、CPU等)，配置面线程通过**ROOT根对象**给所有DATAPLANE下发配置。

NOTE 配置面线程必须等待所有数据面线程初始化完成之后才能启动。

▪ ROOT → DATAPLANE

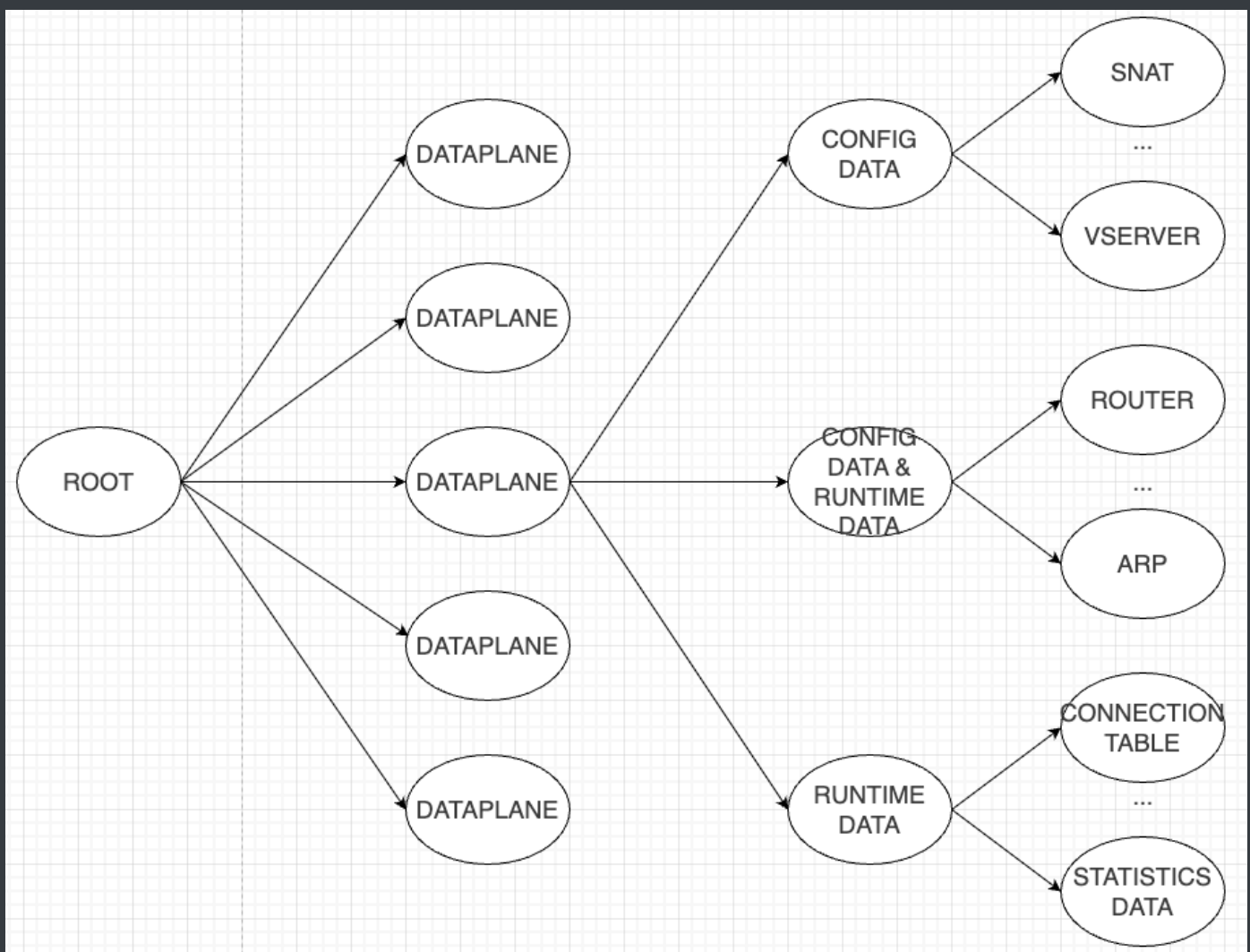
每个数据面线程对应一个 **DATAPLANE** 节点，ROOT 中保存着这些指针。

▪ DATAPLANE 内部结构

每个 DATAPLANE 又进一步拆分为三类数据：

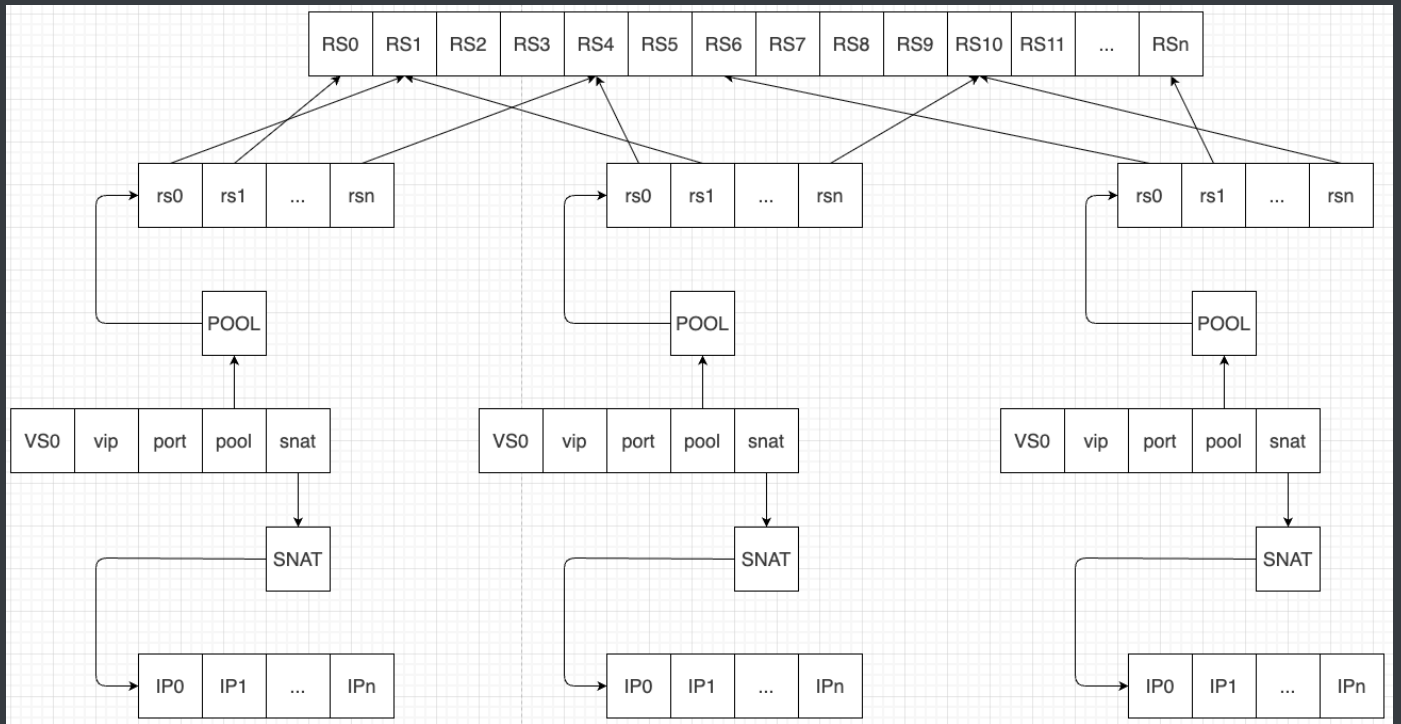
1. **CONFIG DATA**：配置数据，例如 SNAT、VSERVER 等模块的配置信息；
2. **CONFIG DATA & RUNTIME DATA**：部分模块既包含配置也包含运行期状态，例如 ROUTER、ARP；
3. **RUNTIME DATA**：纯运行时数据，例如 CONNECTION TABLE、STATISTICS DATA。

NOTE CONFIG_DATA 是每个 NUMA 独一份还是每个线程一份主要看是否有引用需求，有引用需求(索引)则每个线程一份，没有引用需求则每个 NUMA 一份，CONFIG DATA & RUNTIME DATA 和 RUNTIME DATA 线程私有



二、配置

1、VS配置组织架构图



2、POOL

a、REAL SERVER(RS)

存放在RS池中，最多能够允许20W个RS。

```
struct rserver_stats {
    uint32_t refcnt;
    // TODO
};

struct rserver_mutable {
    uint8_t status; // ONLINE, OFFLINE 暂时不实现
    // TODO
};

struct rserver_base {
    uint16_t port;
    uint32_t id; // RS池中的ID, 便于管理
    struct rserver_stats stats; // 运行时统计信息
    struct rserver_mutable mutable; // 可变的配置信息
};
```

```

struct rserver_v4 {
    uint32_t addr;
    struct rserver_base base;
} __attribute__((aligned(64)));

struct rserver_v6 {
    struct dpdk_ip6_addr addr;
    struct rserver_base base;
} __attribute__((aligned(64))); // 不超过128字节

```

b、POOL 基本结构

存放在POOL池中，最多能够允许1W个POOL

```

struct schedule_algo {
    enum ALGO_TYPE {
        ALGO_POLL,
    } algo_type;
    union {
        uint32_t *next_rs;
    };
};

struct pool_stats {
    // TODO
};

struct pool {
    char *name; // 当前客户使用，内部不再使用name，而是用id，如果脚本要使用，则可以预处理
    struct pool_stats stats;
    struct schedule_algo algo;
    uint32_t id; // 在全局POOL中的索引，便于管理
    uint8_t af_type; // AF_INET AF_INET6
    int rs_count;
};

```

```

    union {
        struct rserver_v4 **rs_v4;
        struct rserver_v6 **rs_v6;
    };
};

```

3、SNAT

a、基本结构

根据配置好的SNAT信息，挂载到 IP_MANAGE 结构中，并发送 ARP/NDP

```

struct snat_base {
    char *name; // 仅仅客户使用，内部把name转换为id，不再使用name,
    uint32_t id; // 在全局SNAT管理结构中的索引
    uint32_t refcnt;
};

struct snat_v4 {
    struct snat_base base;
    uint16_t min_port; // 默认1024，不能是0
    uint16_t max_port; // 默认65535，不能小于port_min
    uint32_t start_addr;
    uint32_t end_addr;
};

struct snat_v6 {
    struct snat_base base;
    uint16_t min_port; // 默认1024，不能是0
    uint16_t max_port; // 默认65535，不能小于port_min
    struct dpdk_ip6_addr start_addr;
    struct dpdk_ip6_addr end_addr;
};

```

4、VS

a、基本结构

struct vs_server_v4存放在线程本地**vs_v4_hash**表中，**vs_v4_hash**也注册到线程私有数据面线程根结构体

struct vs_server_v6存放在线程本地**vs_v6_hash**表中，**vs_v6_hash**也注册到线程私有数据面线程根结构体

```
enum VSERVER_TYPE {
    VSERVER_TCP,
    VSERVER_UDP,
    VSERVER_HTTP,
    VSERVER_HTTPS,
    VSERVER_HTTP2,
    VSERVER_HTTP3,
};

struct vs_server_stats {
    uint32_t refcnt;
};

struct vs_server_mutable {

};

struct vs_server_base {
    char *name;
    uint32_t id; // VS全局ID, 最多1W个
    // forward
    uint8_t af; // AF_INET、AF_INET6
    struct vs_server_stats stats;
    struct vs_server_mutable mutable;
    union {
        struct v4 {
            struct snat_v4 *snat;
            struct pool_v4 *pool;
        };
    };
};
```

```

        } v4;
        struct v6 {
            struct snat_v6 *snat;
            struct pool_v6 *pool;
        } v6;
    };
};

struct vsrver_v4 {
    enum VSERVER_TYPE type;
    uint16_t port;
    uint32_t vip;
    struct vsrver_base base;
};

struct vsrver_v6 {
    enum VSERVER_TYPE type;
    uint16_t port;
    struct dpdk_ip6_addr vip;
    struct vsrver_base base;
};

```

三、TCP & PROXY

状态机与滑动窗口（窗口滑动 & 数据包合并）是 **TCP 协议栈的核心机制**，无论在广域网还是局域网环境中都适用。

在链路稳定、丢包率极低的高速局域网中，**拥塞控制等高级优化机制的重要性会显著降低**，其性能提升作用相对较小，而状态机与滑动窗口仍然是保障可靠传输的关键。

1、TCP状态机

TCP协议栈是TCP状态机驱动的。

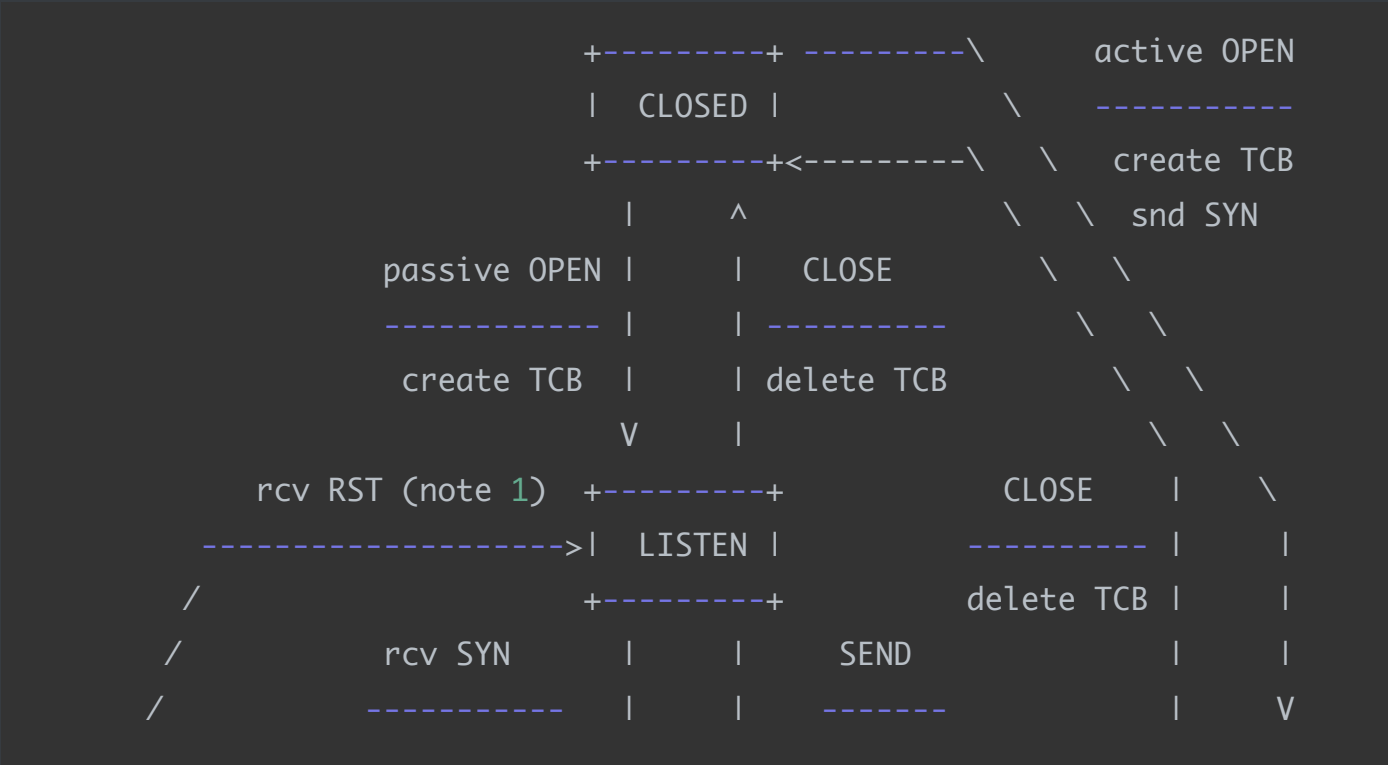
a、TCP状态

TCP共有11种状态，如下所示：

- **ESTABLISHED**：连接已建立，双方可以收发数据；
- **LISTEN**：服务器正在监听端口，等待连接请求(对于我们而言被动打开创建会话表时的状态)；
- **SYN-SENT**：客户端发送了 SYN，等待对方的 SYN+ACK；
- **SYN-RECEIVED**：收到对方 SYN 并回复了 SYN+ACK，等待对方的 ACK；
- **FIN-WAIT-1**：主动关闭方已发送 FIN，等待 ACK 或对方 FIN；
- **FIN-WAIT-2**：已收到对方对 FIN 的 ACK，等待对方 FIN；
- **CLOSE-WAIT**：被动关闭方收到 FIN，并发送ACK，等待本地应用关闭并发送 FIN；
- **LAST-ACK**：被动关闭方发出 FIN 后，等待对方对其 FIN 的 ACK；
- **CLOSING**：双方几乎同时关闭，都发送了 FIN，等待 ACK；
- **TIME-WAIT**：主动关闭方已发送最后 ACK，等待 2×MSL，确保当前连接的所有旧报文消失；
- **CLOSED**：初始/最终状态，表示没有连接存在。

b、状态变换图

状态机条状如图所示：



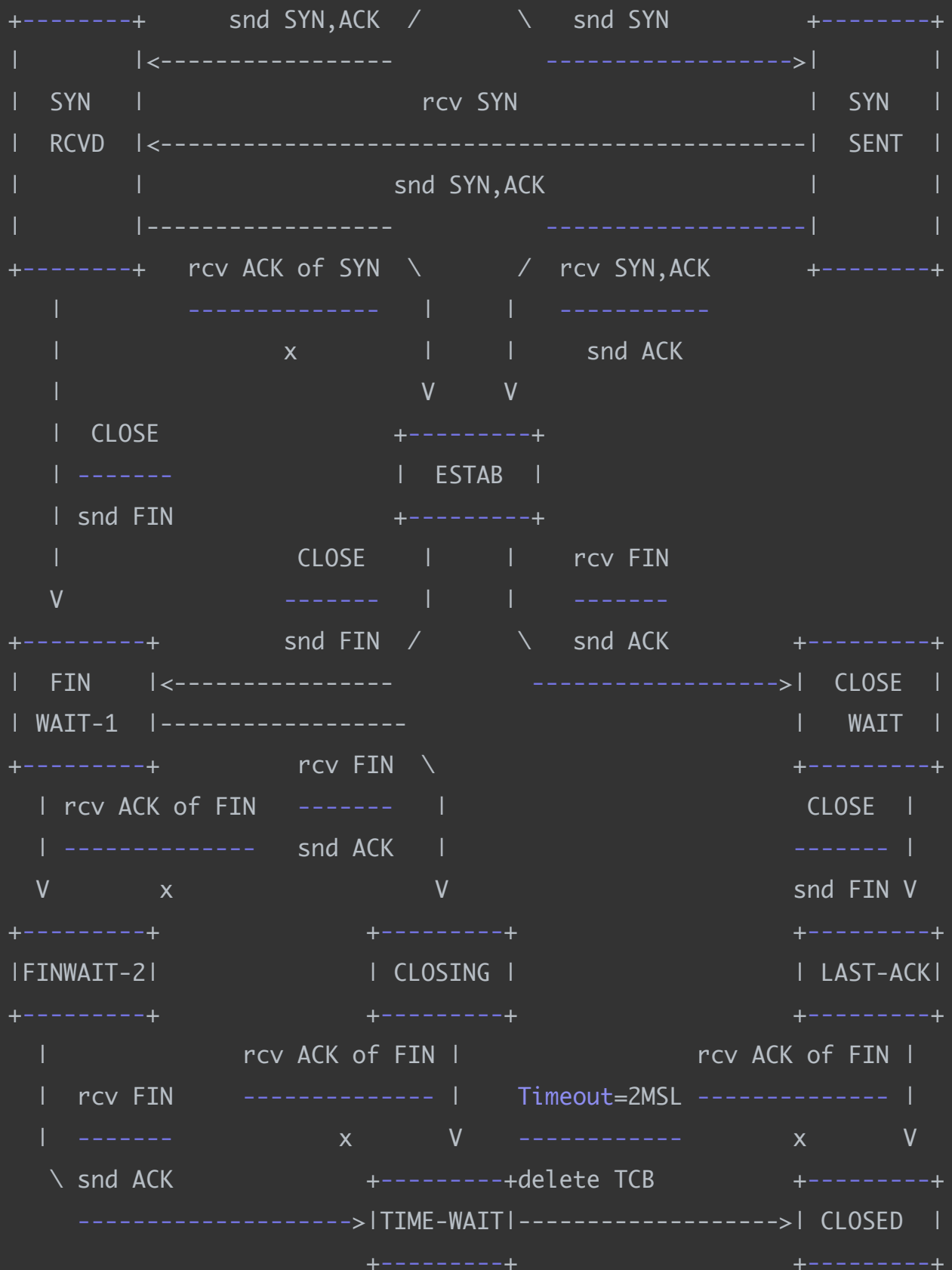
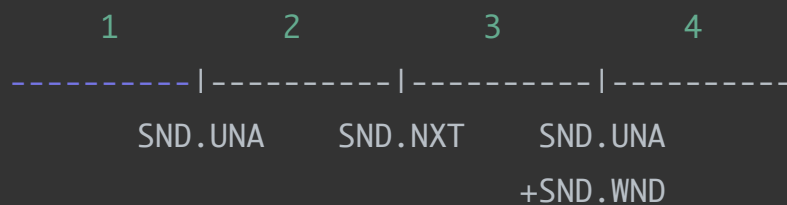


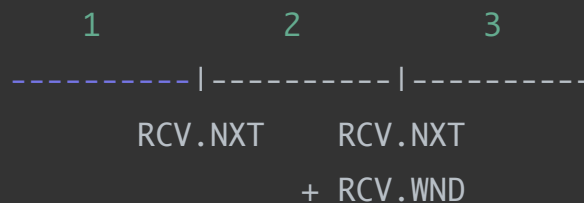
Figure 5: TCP Connection State Diagram

c、发送和接收窗口

- 发送端窗口：



- 接收端窗口：



d、状态机处理基本原则

携带数据的 SYN 报文中，SYN 是第一个序号(协议未禁止SYN携带数据，但是我们实现不允许SYN携带数据)；

携带数据的 FIN 报文中，FIN 是最后一个序号；

当前实现不允许 SYN 和 RST 报文携带数据，FIN报文可以携带数据。

状态机处理的**基本原则**：

(1) 连接不存在 (CLOSED)

- 对于任何传入报文段（除了另一个复位），都要发送复位。
- 不匹配任何现有连接的 SYN 报文会以这种方式被拒绝。

如果传入报文段设置了 **ACK** 位，复位报文的序列号取自该报文的 **ACK 字段**；

否则，复位报文的序列号为 **0**，其 **ACK 字段** 设为：

传入报文段的**序列号**加上其**报文长度**之和。

连接保持在 **CLOSED** 状态。

(2) 非同步状态 (LISTEN、SYN-SENT、SYN-RECEIVED)

- 如果传入报文段确认了尚未发送的数据（即带有不可接受的 ACK），或者其安全级别或隔离区（见附录 A.1）与连接请求的不完全匹配，则发送复位。

复位报文的序列号规则同上：

- 如果传入报文有 ACK 字段 → 用 ACK 字段作为序列号。
- 否则 → 序列号为 0，ACK 字段为报文段序列号与长度之和。

连接保持在相同状态。

(3) 同步状态 (ESTABLISHED、FIN-WAIT-1、FIN-WAIT-2、CLOSE-WAIT、CLOSING、LAST-ACK、TIME-WAIT)

- 对于任何不可接受的报文段（如窗口外序列号或不可接受的确认号），必须回复一个空的确认报文段（不包含用户数据），

该报文段包含：

- 当前发送序列号
 - 以及一个确认号，指示期望接收的下一个序列号
 - 连接保持在原状态。
- 如果传入报文段的安全级别或隔离区与连接请求的不完全匹配，则发送复位并将连接置为 **CLOSED** 状态。

复位报文的序列号取自传入报文段的 **ACK** 字段。

e、状态机处理

为防止处于 **同步状态 (Established)** 的连接受到异常或恶意报文的干扰，TCP 在接收到不符合预期的段时会发送 **Challenge ACK** 进行验证。但为了避免 ACK 风暴：

Challenge ACK 频率必须加以限制：

- 对 **每个连接** 限制为 **每秒不超过 2 个**。

- 当频率超出限制时，直接丢弃异常报文，不再发送 ACK。

```
/**
 * TCP Flags
 */
#define RTE_TCP_CWR_FLAG 0x80 /**< Congestion Window Reduced */
#define RTE_TCP_ECE_FLAG 0x40 /**< ECN-Echo */
#define RTE_TCP_URG_FLAG 0x20 /**< Urgent Pointer field significant */
#define RTE_TCP_ACK_FLAG 0x10 /**< Acknowledgment field significant */
#define RTE_TCP_PSH_FLAG 0x08 /**< Push Function */
#define RTE_TCP_RST_FLAG 0x04 /**< Reset the connection */
#define RTE_TCP_SYN_FLAG 0x02 /**< Synchronize sequence numbers */
#define RTE_TCP_FIN_FLAG 0x01 /**< No more data from sender */

switch (state)
case ESTABLISHED: _tcp_established(); break;
case LISTEN: _tcp_listen(); break;
case SYN_SENT: _tcp_syn_sent(); break;
case SYN_RECEIVED: _tcp_syn_received(); break;
case FIN_WAIT_1: _tcp_fin_wait_1(); break;
case FIN_WAIT_2: _tcp_fin_wait_2(); break;
case CLOSE_WAIT: _tcp_close_wait(); break;
case CLOSING: _tcp_closing(); break;
case LAST_ACK: _tcp_last_ack(); break;
case TIME_WAIT: _tcp_time_wait(); break;
default: _tcp_closed(); break;

////////// ESTABLISHED //////////
_tcp_established:
switch (flags & 0x17)
case ACK:
    if SEG.ACK > SND.UNA && SEG.ACK <= SND.UNA + WIN - 1
        tcp_ack_process(); // SND.UNA = SEG.ACK; 推进窗口
    else if SEG.ACK == SND.UNA
        /*
         * 重复ACK判断
```

```

* 1、ACK接收方仍有未确认的数据；
* 2、传入的ACK不携带数据；
* 3、SYN和FIN位均为关闭状态；
* 4、确认号等于该连接上已收到的最大确认号(等于TCP.UNA)
* 5、传入的ACK中通告的窗口大小等于上一次传入ACK中通告的窗口大小
*/

if 重复ACK
    switch (重复次数)
    case 1:
        if 拥塞窗口允许发送数据
            发送1个新数据包； // 不增加cwnd
    case 2:
        if 拥塞窗口允许发送数据
            发送1个新数据包； // 不增加cwnd
    case 3: // 快速重传
        ssthresh = max(FlightSize / 2, 2 * SMSS);
        cwnd = ssthresh + 3 * SMSS;
        发送丢失的数据包；
    default:
        每收到一个ACK, cwnd += SMSS;
else if 有1字节的数据 & 窗口为0 // 零窗口探测报文
    tcp_send_ack(); // 携带当前窗口
else
    tcl_send_challenge_ack();
    drop();
else
    tcp_send_challenge_ack(); // 避免ACK风暴，内部可以限速
break;
case SYN: case SYN_ACK:
    if SEG.SEQ != RCV.NXT
        tcp_send_challenge_ack();
        drop();
else ESTABLISHED状态下还未传输数据时
    tcp_send_syn_ack(); // 可能是对端未进入同步状态重传的SYN
    drop();
else

```

```

        tcp_send_rst(); // 发送RST
    break;
case FIN: case FIN_ACK:
    if SEG.SEQ == RCV.NXT
        tcp_to_close_wait();
    else if SEG.SEQ > RCV.NXT && SEG.SEQ < RCV.NXT + RCV.WIN
        tcp_send_ack(); // 触发快速重传
        进入滑动窗口，进行序号合并，等待前面的数据处理完后再处理FIN;
    else
        tcp_send_challenge_ack();
    break;
case RST_ALL: // RST RST_ACK RST_FIN RST_SYN RST_FIN_ACK RST_SYN_FIN
RST_SYN_ACK RST_SYN_FIN_ACK
    if SEG.SEQ == RCV.NXT
        丢弃接收和发送队列以及重试队列中数据;
        关闭连接;
        向代理的另一端报告连接被复位;
    else
        tcp_send_challenge_ack();
    break;
case SYN_FIN: case SYN_FIN_ACK:
    tcp_send_challenge_ack(); // 这种错误的包既可以发送RST也可以发送
challenge ack，因为对端状态可能异常，但是也存在是中间设备修改数据导致
drop();
    break;
default: // 无标记报文
    if ! (SEG.SEQ >= RCV.NXT + RCV.WIN && SEG.SEQ + SEQ.LEN - 1 <
RCV.NXT)
        tcp_send_ack();
    else
        tcp_send_challenge_ack();
    break;

```

```

//////////////////// LISTEN //////////////////////

```

```

_tcp_listen: // 未命中会话表的数据包处理方式，如果有内核就丢给内核，没有内核就按照下面的方式处理...
switch (flags & 0x17)
case SYN:
    创建会话表; // 创建会话表在之前处理
    /*
        * ISN = (M + F(localhost, localport, remotehost, remoteport,
secret_key))
        * M和时间相关 当前毫秒数 * 4
        */
    选择 IRS;
    tcp_send_syn_ack();
    进入 SYN-RECEIVED 状态;
    break;
case ACK: case SYN_FIN_ACK: case SYN_ACK: case FIN_ACK:
    tcp_send_rst();
    break;
default: // 即含有ACK也含有RST的也是直接丢弃，因为RST的检查优先级更高，RST的行为就是丢弃
    drop();
    break;

//////////////////// SYN-SENT //////////////////////
_tcp_syn_sent:
switch (flags & 0x17)
case SYN_ACK:
    if SEG.ACK == SND.NXT
        tcp_send_ack();
        进入 ESTABLISHED 状态
    else
        tcp_send_rst();
        drop();
    break;
case SYN: // 同时打开
    if 是旧的 SYN 报文段 then

```



```

        发送 RST 报文
    else
        tcp_send_syn_ack();
        进入 SYN_RECEIVED 状态;
    break;
case ACK:
    tcp_send_rst();
    drop();
    break;
case RST_ACK: case RST_SYN_ACK: case RST_SYN_FIN_ACK: case
RST_FIN_ACK:
    if SEG.ACK == SND.NXT // SYN报文、RST报文不允许携带数据(协议允许, 实现不
    允许)
        tcp_conn_close();
        drop();
    else
        drop();
    break;
default: // 无控制位
    drop();
    break;

//////////////////// SYN-RECEIVED //////////////////////
_tcp_syn_received:
switch (flags & 0x17)
case ACK:
    if SEG.ACK == SND.NXT
        进入 ESTABLISHED 状态
    else
        tcp_send_rst();
    break;
case SYN:
    if SEG.SEQ == RCV.NXT // SYN + ACK 丢失了会导致 SYN 重发
        tcp_send_syn_ack();
    else // 攻击 SYN 或者

```

```

        tcp_send_challenge_ack();
    break;
case RST_ALL:
    if SEG.SEQ == RCV.NXT
        if 本地是服务端
            进入 LISTEN 状态，对我们来说就是关闭；
        else
            进入 CLOSE 状态；
    else
        drop(); // 静默丢弃
    break;
default:
    tcp_send_ack();
    break;

//////////////////// FIN-WAIT-1 //////////////////////
_tcp_fin_wait_1:
switch (flags & 0x17)
case ACK:
    if SEG.ACK == SND.NXT
        进入 FIN-WAIT-2 状态
    else
        tcp_send_challenge_ack();
    break;
case FIN_ACK:
    if SEG.SEQ == RCV.NXT && SEG.ACK == SND.NXT
        tcp_send_ack();
        进入 TIME-WAIT 状态
    else
        tcp_send_challenge_ack();
        break;
case RST:
    if SEG.SEQ == RCV.NXT
        进入 TIME-WAIT 状态
    else

```

```

        tcp_send_challenge_ack();
    break;
case RST_ACK:
    if SEG.SEQ == RCV.NXT && SEG.ACK == SND.NXT
        进入 TIME-WAIT 状态
    else
        tcp_send_challenge_ack();
    break;
default:
    tcp_send_challenge_ack();
    break;

//////////////////// FIN-WAIT-2 //////////////////////
_tcp_fin_wait_2:
switch (flags & 0x17)
case FIN:
    if SEG.SEQ == RCV.NXT // 对端正常关闭
        tcp_send_ack();
        进入 TIME-WAIT状态;
    else
        tcp_send_challenge_ack();
    break;
case FIN_ACK:
    if SEG.SEQ == RCV.NXT && SEG.ACK == SND.NXT // 重传
        tcp_send_ack();
        进入 TIME-WAIT状态;
    else
        tcp_send_challenge_ack();
    break;
case RST:
    if SEG.SEQ == RCV.NXT
        进入 TIME-WAIT 状态;
    else
        tcp_send_challenge_ack();
    break;

```

```

case RST_ACK:
    if SEG.SEQ == RCV.NXT && SEG.ACK == SND.NXT
        进入 TIME-WAIT 状态
    else
        tcp_send_challenge_ack();
    break;
default:
    tcp_send_challenge_ack();
    break;

//////////////////// CLOSE-WAIT //////////////////////
_tcp_close_wait:
switch (flags & 0x17)
case FIN: // 重传
    if SEG.SEQ == RCV.NXT
        tcp_send_ack();
    else
        tcp_send_challenge_ack();
    break;
case FIN_ACK:
    if SEG.SEQ == RCV.NXT && SEG.ACK == SND.NXT
        tcp_send_ack();
    else
        tcp_send_challenge_ack();
    break;
case RST:
    if SEG.SEQ == RCV.NXT
        tcp_conn_close();
    else
        tcp_send_challenge_ack();
    break;
case RST_ACK:
    if SEG.SEQ == RCV.NXT && SEG.ACK == SND.NXT
        tcp_conn_close();
    else

```

```

        tcp_send_challenge_ack();
    break;
default:
    tcp_send_challenge_ack();
    break;

////////// CLOSING //////////
_tcp_closing:
switch (flags & 0x17)
case FIN:
    if SEG.SEQ == RCV.NXT - 1 // 重传报文段
        tcp_send_ack();
    else
        tcp_send_challenge_ack();
    break;
case ACK:
    if SEG.ACK == SND.NXT
        进入TIME-WAIT状态; // 同时关闭, 都进入TIME-WAIT状态
    else
        tcp_send_challenge_ack();
    break;
case RST:
    if SEG.SEQ == RCV.NXT
        进入TIME-WAIT状态
    else
        tcp_send_challenge_ack();
    break;
case RST_ACK:
    if SEG.SEQ == RCV.NXT && SEG.ACK == SND.NXT
        进入TIME-WAIT状态
    else
        tcp_send_challenge_ack();
    break;
default:
    tcp_send_challenge_ack();

```

```

        break;

////////// LAST-ACK //////////
_tcp_last_ack:
switch (flags & 0x17)
case ACK:
    if SEG.SEQ == RCV.NXT
        tcp_conn_close();
    else
        tcp_send_challenge_ack();
    break;
case RST:
case RST_ACK:
    if SEG.SEQ == RCV.NXT
        tcp_conn_close();
    else
        tcp_send_challenge_ack();
    break;
default:
    tcp_send_challenge_ack();
    break;

////////// TIME-WAIT //////////
_tcp_time_wait:
switch (flags & 0x17)
case FIN: // 可能是前一个FIN丢失了
case FIN_ACK:
    if SEG.SEQ == RCV.NXT
        tcp_send_ack();
    else
        tcp_send_challenge_ack();
    break;
case RST: // RFC 1337 TIME-WAIT Assassination Hazards in TCP 专门讨论了
这个问题，忽略是最好的方法

```

```

case RST_ACK:
    drop(); // 忽略
    break;
default: // 没有标记的数据包
    tcp_send_challenge_ack();
    break;

////////// CLOSED //////////
_tcp_closed: // 控制块(TCB)不存在
switch (flags & 0x17)
case RST_ALL:
    drop();
    break;
default:
    tcp_send_rst();
    break;

```

f、序号判断

窗口最大1GB，为防止序号回绕导致比较异常，采用如下接口比较

```

static INLINE bool tcp_seq_before(uint32_t a, uint32_t b)
{
    return (int32_t)(a - b) < 0;
}

static INLINE bool tcp_seq_after(uint32_t a, uint32_t b)
{
    return (int32_t)(a - b) > 0;
}

static INLINE bool tcp_seq_leq(uint32_t a, uint32_t b)
{
    return (int32_t)(a - b) <= 0;
}

```

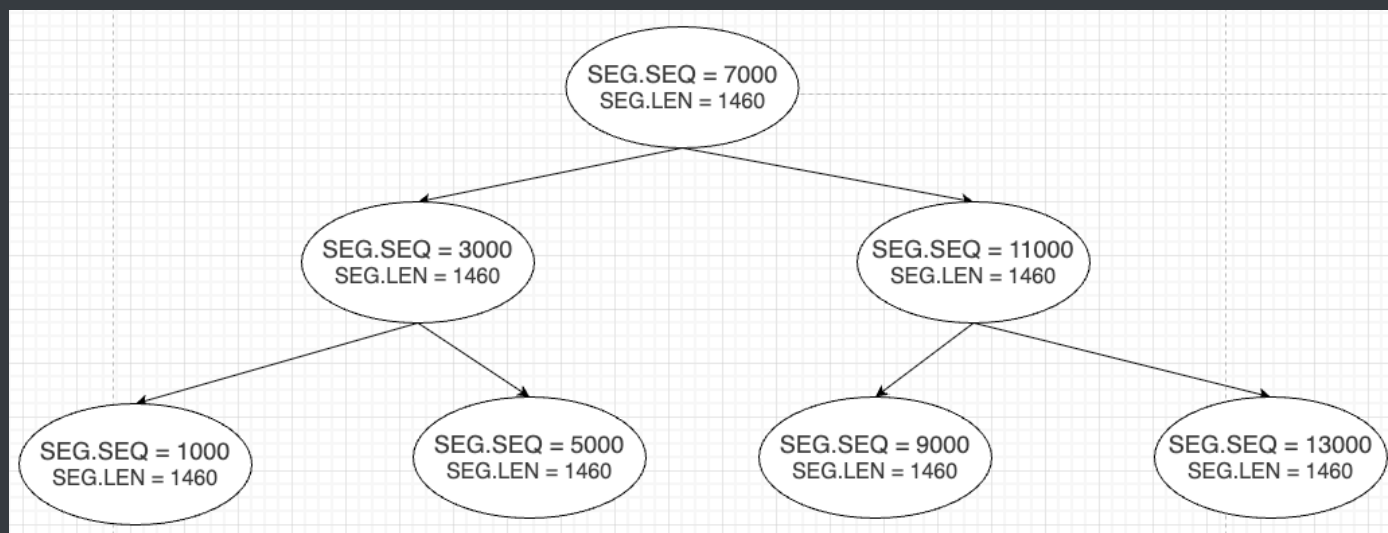
```
static INLINE bool tcp_seq_geq(uint32_t a, uint32_t b)
{
    return (int32_t)(a - b) >= 0;
}

// 不对成边界，左边界大于等于，右边界小于
static INLINE bool tcp_seq_between(uint32_t x, uint32_t start,
uint32_t end)
{
    return tcp_seq_geq(x, start) && tcp_seq_before(x, end);
}
```

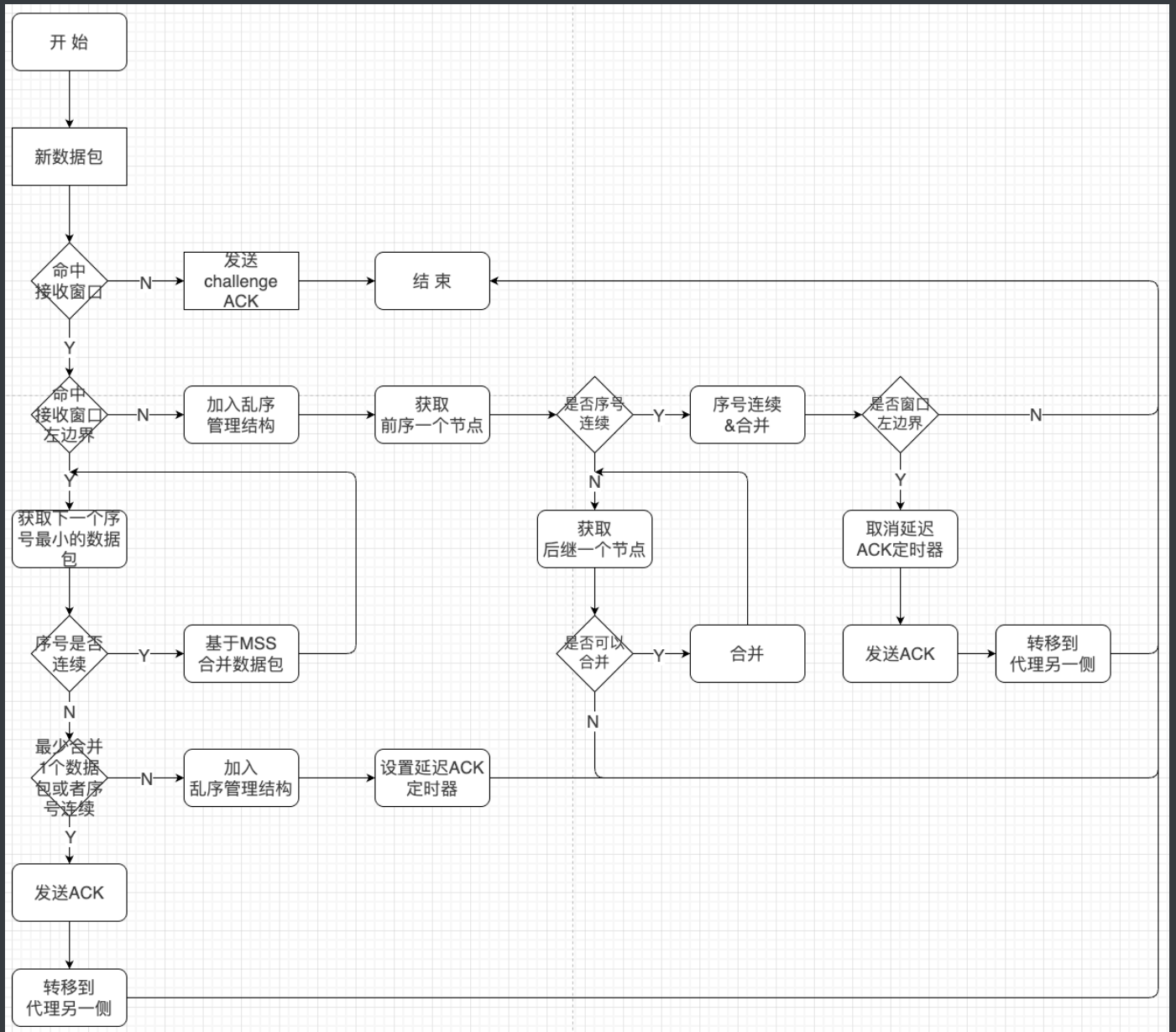
2、滑动窗口(数据包合并)

数据包合并的必要性：当TCP超时并重传时，它不一定要重传同样的报文段，TCP协议允许进行重新分组而发送一个较大的报文段，这将有助于提高性能，TCP使用字节序号而不是报文段序号来进行识别它要发送的数据和进行确认。

采用红黑树来实现乱序数据包的合并，如下是一个滑动窗口内的乱序数据包：



在握手完成之后，同步状态下数据包处理行为如下图所示：



3、TCP的超时与重传

TCP 在设计上始终期望以理想状态进行数据传输，以此保持尽可能高的传输速率。然而，实际网络环境具有实时变化和不确定性，链路带宽、延迟和丢包率都可能动态波动。

为尽量逼近这种理想状态并避免网络拥塞崩溃，TCP 采用了一系列拥塞控制与恢复机制：

- **慢启动 (Slow Start)**：在连接初期或超时重传后，从较小的拥塞窗口开始，指数级增加发送速率，以快速探测可用带宽。
- **拥塞避免 (Congestion Avoidance)**：在接近可用带宽时，线性增长窗口，细致调整以维持稳定传输。
- **快速重传 (Fast Retransmit) 与快速恢复 (Fast Recovery)**：通过检测重复 ACK 提

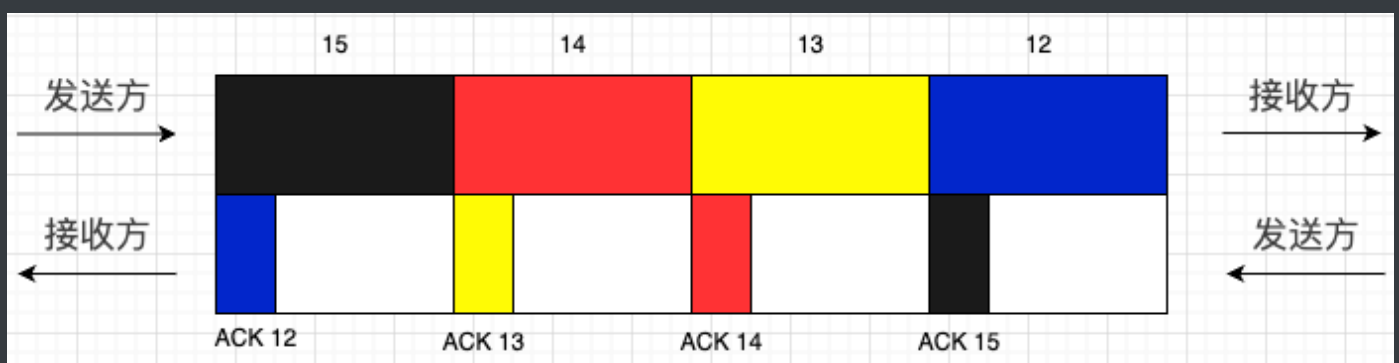
前重传丢失数据，并在恢复后避免窗口回退过度。

这些机制协同工作，使 TCP 能够在不断变化的网络环境中**动态调整发送速率**，既追求高吞吐，又防止拥塞崩溃，从而在广域网与局域网环境中保持高效、稳定的数据传输。

a、TCP传输的理想状态

发送方和接收方之间的管道被填满，此时不论拥塞窗口和通告窗口是多少，它都不能容纳更多的数据。每当接收方在某一个时间单位从网络上移去一个报文段，发送方再发送一个报文段到网络上，但是不管有多少报文段填充了这个管道，返回路径上总是有相同数目的ACK，这就是连接的理想状态。

发送的理想状态如下图所示(这个图是理解后面拥塞算法的基础，任何拥塞算法都期望达到这种理想状态)：



b、慢启动和拥塞避免

这里算法以数据丢失作为拥塞的信号，显示拥塞通知(ECN)作为拥塞信号暂不支持。

基本概念：

拥塞窗口(cwnd)：发送端侧的限制，规定在接收到确认(ACK)之前，发送端可以向网络中注入的数据量上限。

接收端通告窗口(rwnd)：接收端侧的限制，规定接收端可以接收的未确认数据量上限。

cwnd和rwnd的最小值共同决定发送数据的上限。

i、Reno

▪ 初始拥塞窗口(IW)

若 $SMSS > 2190$ 字节: (SMSS: 发送端MSS)

$IW = 2 * SMSS$, 且不得超过2个报文段

若 $1095 < SMSS \leq 2190$ 字节:

$IW = 3 * SMSS$, 且不得超过3个报文段

若 $SMSS \leq 1095$ 字节:

$IW = 4 * SMSS$, 且不得超过4个报文段

拥塞窗口和接收窗口都是握手完成之后的概念, 如果在握手阶段丢包, 则 $IW = 1$ 。

▪ 慢启动阈值(ssthresh)

初始ssthresh设置rwnd

当 $cwnd < ssthresh$ 时使用慢启动;

当 $cwnd > ssthresh$ 时使用拥塞避免;

当 $cwnd = ssthresh$ 时, 发送端可任选慢启动或拥塞避免。

▪ 慢启动阶段cwnd增长

每当收到一个累计确认新数据的ACK时, TCP最多将cwnd增加SMSS字节。

$cwnd += \min(N, SMSS)$ # N 是本次 ACK 字节数

慢启动结束条件: 1、cwnd超过ssthresh(或等于ssthresh), 或2、检测到拥塞。

▪ 丢包与ssthresh更新

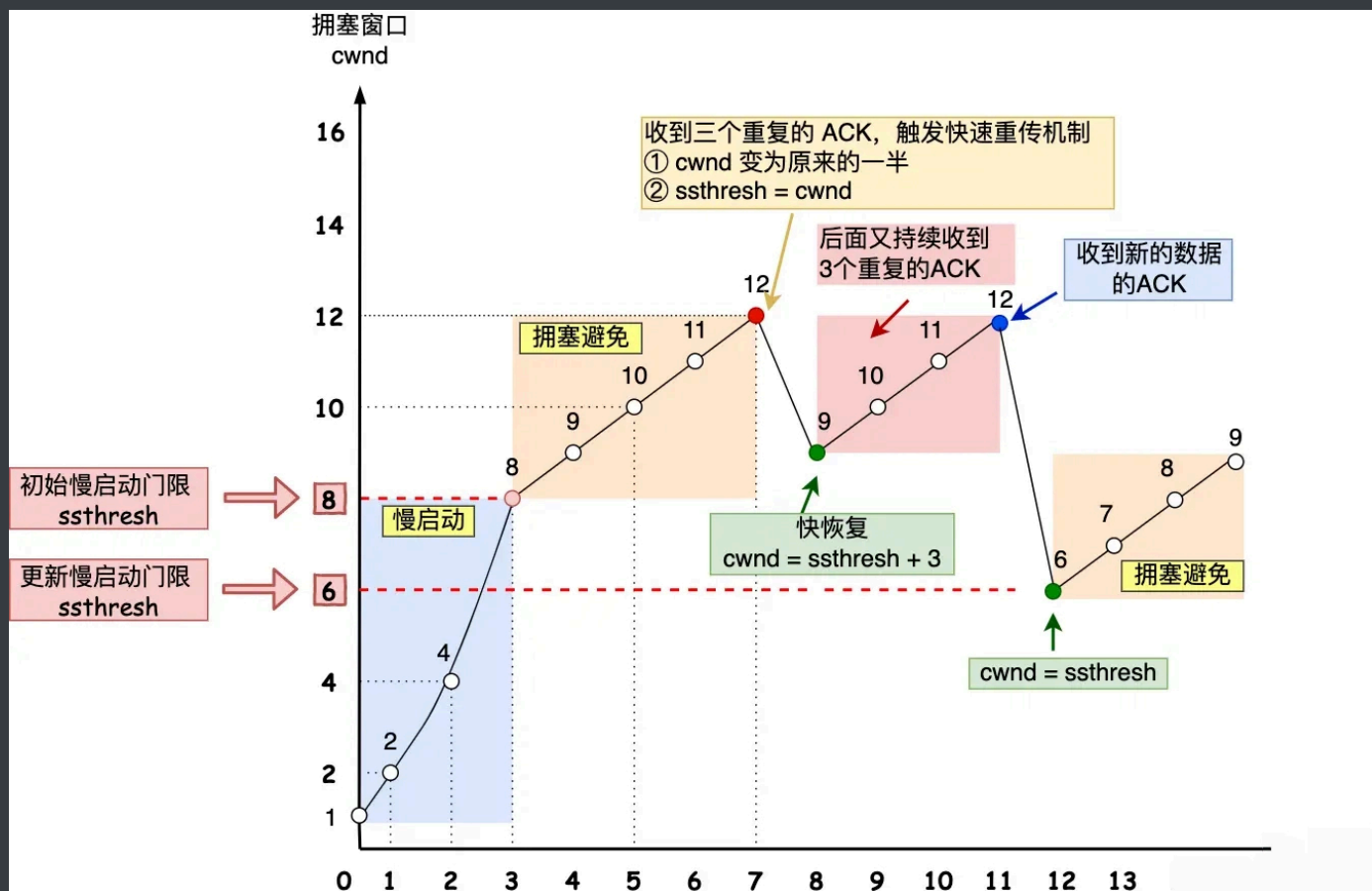
当通过重传定时器检测到丢包且丢失段尚未重传过时, ssthresh必须按下式更新:

$ssthresh = \max(FlightSize / 2, 2 * SMSS)$ # FlightSize 是网络中已发送但未确认的数据量。

在超时后, 拥塞窗口cwnd必须为1。

▪ Reno算法示意图

下图是Reno算法示意图：



TCP拥塞控制Reno算法

ii、CUBIC

Reno在高带宽、长RTT网络中增长太慢。CUBIC与RTT无关，更快利用带宽，又避免过度拥塞。

CUBIC 已经被Linux、Windows和Apple的TCP协议栈采用为默认的拥塞控制算法。

CUBIC 默认慢启动推荐使用HyStart++(暂不支持)，使用Reno 慢启动

CUBIC 在拥塞避免阶段用 **时间驱动的三次函数** 调整窗口，算法如下(要理解这个算法需要了解BIC二分搜索来找当前链路适合的拥塞算法，这里就不展开了，当前只需要这么计算就行)：

$$W_{\text{cubic}}(t) = C \cdot (t - K)^3 + W_{\text{max}} \quad (1)$$

$$K = \sqrt[3]{\frac{W_{\max}(1 - \beta_{\text{cubic}})}{C}} \tag{2}$$

符号	含义	典型取值 / 推荐值
C	增长曲线陡峭度（scaling constant），控制窗口增长速度	0.4 （RFC 8312 推荐）
β_cubic	缩减系数，丢包时 cwnd 会缩到 β·W_max	0.7 （RFC 8312 推荐）
W_max	上一次丢包（拥塞）时的拥塞窗口大小	动态记录，无固定值
t	从上一次丢包后计时的秒数	实时变量
K	时间拐点，使得 t=K 时 W_cubic= W_max	由上式计算得到
W_cubic(t)	t 时刻目标拥塞窗口	算式输出

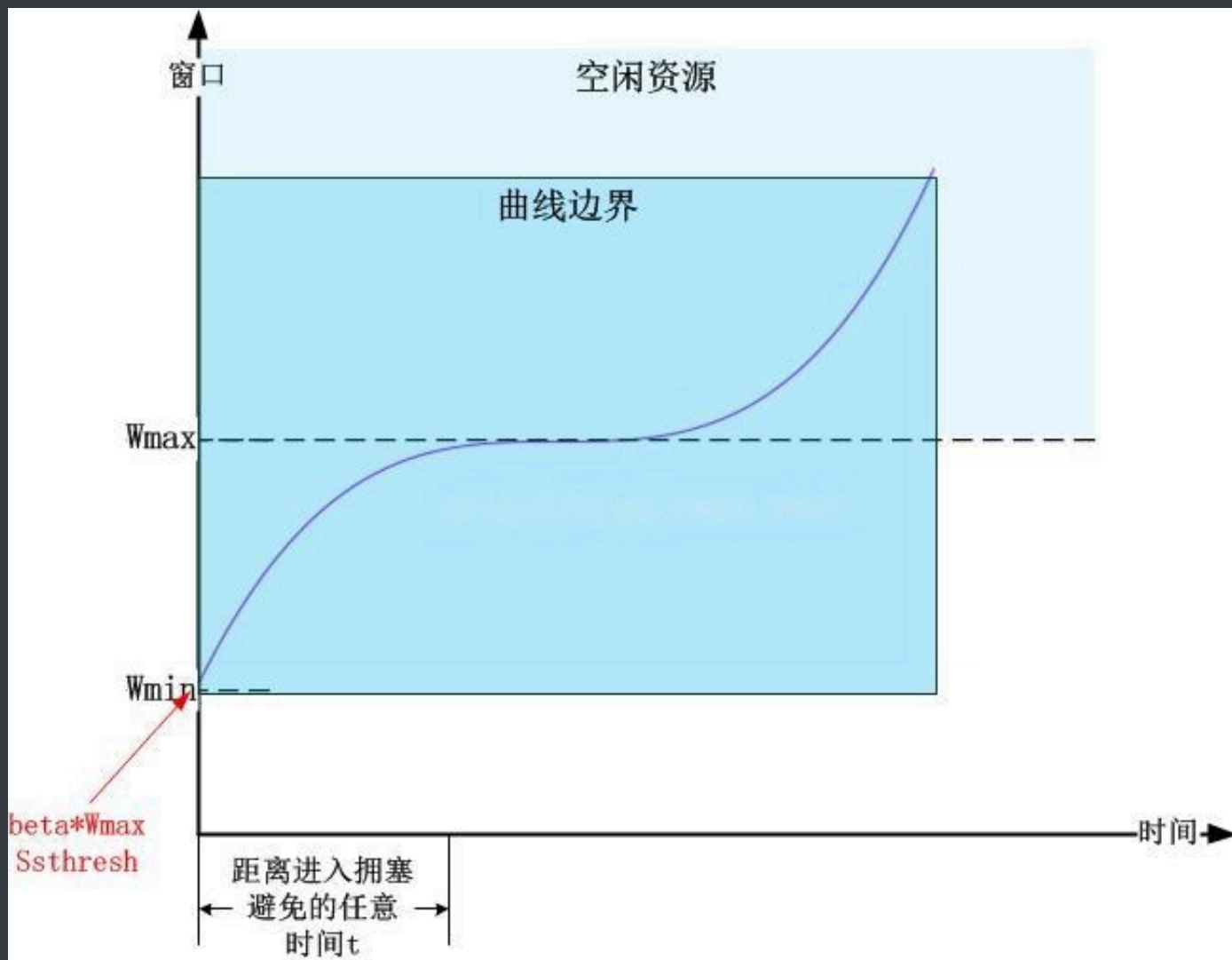
当 t = 0 时：

$$\begin{aligned}
 W_{\text{cubic}}(t) &= C \cdot (-K)^3 + W_{\max} \\
 &= C \cdot \left(-\frac{W_{\max}(1 - \beta_{\text{cubic}})}{C}\right) + W_{\max} \\
 &= W_{\max} \cdot -(1 - \beta_{\text{cubic}}) + W_{\max} \\
 &= W_{\max} \cdot (-0.3) + W_{\max} \\
 &= 0.7 W_{\max}
 \end{aligned} \tag{3}$$

当 t = K 时：

$$W_{\text{cubic}}(t) = W_{\max} \tag{4}$$

CUBIC算法示意图：



CUBIC算法

c、快速重传和快速回复

当接收端接收到乱序到达的报文段时，TCP接收端应该立即发送一个重复ACK(延迟ACK只对顺序到达的报文)。

当接收端收到的报文段填补了序列号空间中的全部或部分缺口时，接收端应当立即发送一个ACK。

- 收到第1、2个重复ACK时，若接收端的通告窗口允许， $\text{FlightSize} \leq \text{cwnd} + 2 * \text{SMSS}$ 且有新数据可以发送，发送端应该发送1个未发送过的数据段，期望尽快触发快速重传；
- 收到第3个重复ACK时，设置 ssthresh ；
- 重传从 SND.UNA 开始丢失段，设置 $\text{cwnd} = \text{ssthresh} + 3 * \text{SMSS}$ ；
- 收到第4个即其后的重复ACK时，每收到一个额外的重复ACK， cwnd 必须增加 SMSS ；

- 在cwnd和接收窗口允许的情况下，当有未发送过的数据时，TCP应当发送 $1 * \text{SMSS}$ 的数据；
- 当收到第一个确认先前未确认数据的ACK时，将cwnd设置为sssthresh，快速恢复阶段结束，进入拥塞避免阶段。

d、重启空闲连接

TCP在相对长的一段时间后(一个RTO)保持空闲后应使用 **慢启动** 重新开始传输。

e、nagle算法

在发送端默认启用nagle算法，算法逻辑如下所示(必须实现，但可以关闭，在局域网中可以不使用该算法)：

```
if 有未确认小数据在飞行
    if 待发送数据小于MSS
        缓存小数据
    else
        发送数据
else
    立即发送数据
```

4、TCP选项

a、必须支持的选项：

- kind = 0 TCP选项的结束

```
0
0 1 2 3 4 5 6 7
+-+--+--+--+--+--+
|           0           |
+-+--+--+--+--+--+
```

- kind = 1 每个选项结束可以用于对齐

```

0
0 1 2 3 4 5 6 7
+---+---+---+---+---+---+
|           1           |
+---+---+---+---+---+---+

```

- kind = 2 MSS 最大报文段，只在SYN <+ ACK> 报文段中使用

kind = 2

length = 4

中间路由器如果发现SYN或SYN+ACK包中MSS大于本地MSS，那么路由器可以修改SYN或SYN+ACK的MSS为当前MSS(不是所有路由器都支持)。

```

0                               1                               2                               3
0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1
+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+
|           2           | Length | Maximum Segment Size (MSS) |
+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+

```

b、可选支持

- kind = 3 窗口缩放因子

双方都必须在其报文段中发送窗口缩放选项才能在任意方向上启用窗口缩放(那个方向启用了那个方向就支持窗口缩放)。

最大窗口缩放因子是14。

```

+-----+-----+-----+
| Kind=3 | Length=3 | shift.cnt |
+-----+-----+-----+
          1           1           1

```

窗口缩放的锚点：另一端消耗数据的速度，即另一侧的接收端ACK一个数据后，本端的接收窗口就增大。

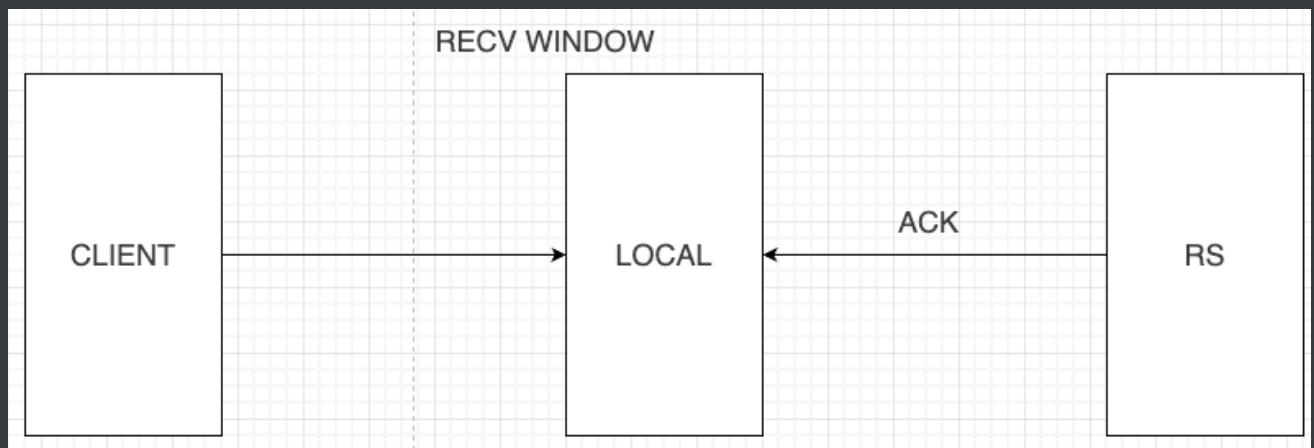
在客户端与代理建立连接时，代理初始通告的接收窗口默认为 **5 MB**。

在代理与真实服务器完成握手后：

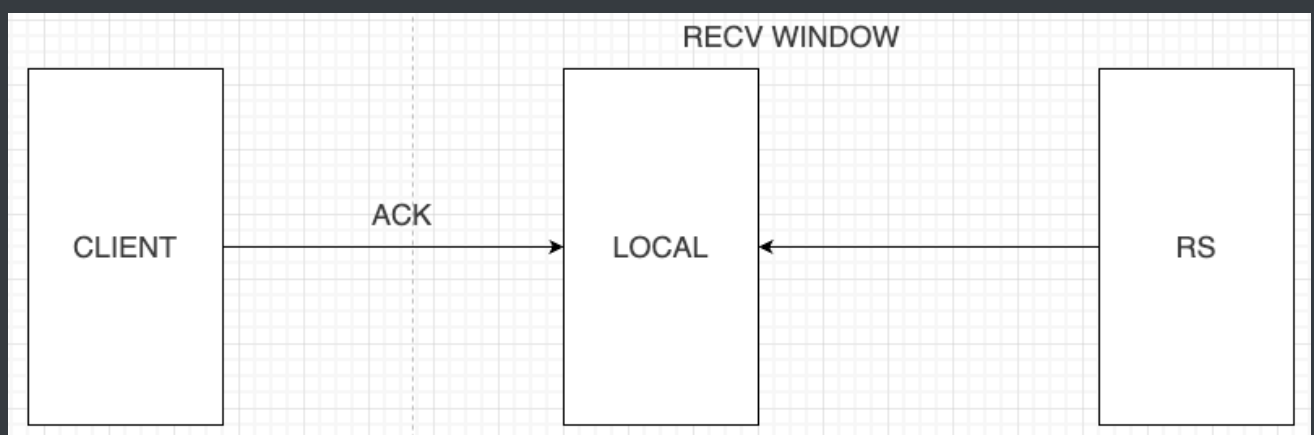
- 如果真实服务器的可用接收窗口 **小于 5 MB**，代理会将通告给客户端的实际窗口**缩小到与真实服务器一致的大小**；
- 如果真实服务器的接收窗口 **大于或等于 5 MB**，代理则**固定使用 5 MB 作为通告窗口**进行数据传输。
- 反方向接收窗口亦如此(代理客户端接收窗口不超过5MB)。

如下图所示：

面向CLIENT的RECV(接收端)窗口随着接收客户端数据而窗口缩小，随着**RS的ACK**而窗口扩大。



面向RS的RECV(接收端)窗口随着接收RS发送的数据窗口缩小，随着接收**CLIENT的ACK**而窗口扩大。



窗口扩大，即右边界向右移动，当窗口增大2个MSS或者缓存区的一半时(取较小值)，单独发送窗口通告(也可以让零窗口快速启动)，或者由ACK报文携带新窗口。

- kind = 4 选择ACK标记(暂时不支持)

```

+-----+-----+
| Kind=4 | Length=2 |
+-----+-----+

```

- kind =5 选择ACK(暂时不支持)

```

+-----+-----+-----+-----+
| Kind=5 | Length | Left Edge of 1 | Right Edge of 1 |
+-----+-----+-----+-----+
|          ... (可携带多个 SACK 块, 每块 8 字节)          |

```

- kind = 8 时间戳选项

一旦协商成功，整个连接期间每个非RST报文段必须携带时间戳选项，RST报文段应该携带。

如果收到一个非RST报文段但未携带时间戳选项，则应该丢弃该报文段。TCP不得因为缺少预期的时间戳选项而中止连接（毫秒）。

```

+-----+-----+-----+-----+
| Kind=8 | 10 | TS Value (TSval) | TS Echo Reply |
| (TSecr) |
+-----+-----+-----+-----+
|          1          1          4          4          |

```

- kind = 34 **TCP Fast Open (TFO) RFC 7413** 允许在三次握手未完成时携带数据(暂时不支持)

5、定时器

TCP定时器触发粒度是10ms，即每10ms扫描一次是否有可以触发的定时器，避免定时器扫描过多从而占用CPU耗时。

a、零窗口探测定时器(persist timer)

当发送端接收到 **窗口为 0** 的通告时，启动 **Persist Timer**。

1、初始超时：**5 s**

2、后续退避：**6 s → 12 s → 24 s → 48 s → 60 s**（60 s 为上限，后续保持 60 s）。

3、由于接收端一旦窗口增大就会主动通告，所以探测间隔较长不会影响数据传输。

零窗口探测（Zero Window Probe）

探测报文通常携带 **1 字节数据**（可为有效数据，也可为占位数据）以测试窗口是否开放。

b、重传定时器

重传定时器分为三部分：RTT测量、重传定时器以及RTO算法

i、RTT 测量

在 TCP 中，重传定时器（RTO）依赖 RTT 测量来动态调整。

按照协议约定：**每一个 RTO 周期内测量一次 RTT**。

在高速局域网环境中，RTO 可能仅为微秒级，这意味着一个连接每秒可能进行上千次 RTT 采样，这本身可能成为潜在的性能开销点。

ii、重传定时器

初始值：RTO 初始设为 **1 秒**（RFC 6298 推荐值）。

动态调整：根据实时测得的 RTT 平滑计算更新 RTO。

指数退避：当发生重传时，每次超时重传的 RTO 以 2 为倍数递增：

- 第 1 次超时： $1 \times \text{RTO}$
- 第 2 次超时： $2 \times \text{RTO}$
- 第 3 次超时： $4 \times \text{RTO}$
- ...以此类推。

- 上限：RTO 最大不超过 120 秒**（防止长时间重传阻塞资源）。

iii、RTO算法

```
# 第一次测量
SRTT = RTT
RTTVAR = RTT / 2
RTO = SRTT + max(G, K * RTTVAR); # SRTT + max(1, 4 * RTTVAR);

# 第二次测量
RTTVAR = (1 - 0.25) * RTTVAR + 0.25 * |SRTT - R'|
SRTT = (1 - 0.125) * SRTT + 0.125 * R'
RTO = SRTT + max(G, K * RTTVAR)
```

c、延迟ACK定时器

延迟 ACK 定时器默认延迟 **20 ms**。在这段时间内如果没有新的、连续的按序数据段到达，接收端就会立即对当前已接收的最大连续序列进行 ACK，并推进接收窗口左边界。

延迟 ACK 的默认策略是**每两个连续报文确认一次**，以减少 ACK 数量。

延迟 ACK 仅适用于连续、按序的数据流。

对于**乱序数据**，必须立即发送 ACK：

- 例如：期望序号为 **1001**，却先收到了 **1501**，应立即发送 **重复 ACK=1001** 提示发送端重传缺失数据。
- 随后缺失段 **1001** 到达，但没有再接收到下一个顺序段时，应**立即**发送 ACK 更新累计确认，而不是继续延迟。

d、TCB超时定时器

TCP 超时定时器与会话表使用同一套超时机制。

当连接进入 **TIME-WAIT** 状态时，不单独设置额外的 TIME-WAIT 定时器，而是直接将其超时管理交由会话表的通用超时机制处理。

TIME-WAIT 状态下不能立即销毁会话表项：

- TIME-WAIT 的存在是为了处理可能延迟到达的旧报文，避免与后续新连接混淆。
- 保留会话表项也便于未来扩展，例如允许在 **TIME-WAIT** 状态下对合法 **SYN** 包执行快速握手。

6、协议间联动

a、ICMP / ARP(NDP) 与 TCP 的联动

- TCP→ARP/NDP（超时上报）

当同一连接发生**多次重传超时（RTO）**时，TCP 应上报给 ARP/NDP 层，用于触发邻居解析重试或标记链路/下一跳可疑（降级、加速重查等）。

- ICMP→TCP（错误上报）

收到与该连接匹配的 ICMP 差错报文时，应反馈给 TCP：

端口不可达（Port Unreachable）

主机不可达（Host Unreachable）

网络不可达（Network Unreachable）

需要分片但 DF 置位（Frag Needed, DF set）

TTL/ Hop Limit 过期等

由 TCP 做相应处理（关闭、报错、路径 MTU 调整等）。

b、实现状态

以上机制**接口预留、暂不实现**；待主线功能稳定后补齐，PMTU需要ICMP反馈，故也暂不支持。

四、其他

1、会话表

a、基本结构

```
struct conn_hdr4 {
    uint32_t sip;
    uint32_t dip;
    uint32_t sport;
    uint32_t dport;
    uint32_t version;
} __attribute__((aligned(64)));

struct conn_hdr6 {
    struct dpdk_ip6_addr sip;
    struct dpdk_ip6_addr dip;
    uint32_t sport;
    uint32_t dport;
    uint32_t version;
} __attribute__((aligned(64)));

struct tcb { // 传输控制块
    uint8_t recv_win_shift;
    uint8_t send_win_shift;
    uint16_t mss; // 发送端SMSS
    uint32_t cwnd; // 拥塞窗口
    uint32_t ssthresh; // 拥塞阈值
    struct {
        uint32_t srtt; // 微妙
        uint32_t rttvar; // 微妙
        uint32_t rtt_stamp; // 测量时间戳
        uint32_t rto; // 微妙 [200ms ~ 60000ms]  计算结果 * 0.001
    }; // rto测量

    struct recv_window {
        uint32_t next;
        uint32_t window;
        uint32_t irs; // init recv seq
    } recv_win;
```

```

struct send_window {
    uint32_t unack;
    uint32_t next;
    uint32_t window;
    uint32_t wl1; // send window last update sequence
    uint32_t wl2; // send window last update acknowledgement
    uint32_t iss; // init send seq
} send_win;

uint8_t rtx_r1; // 重传次数 超过这么多次就向IP层报告以触发网关诊断
uint8_t rtx_r2; // 重传超时时间 超过则关闭连接
};

struct conn_body {
    uint8_t side; // CLIENTSIDE, SERVERSIDE
    uint8_t state; // TCP状态
    uint8_t af; // AF_INET, AF_INET6,
    uint8_t port; // 出口 是否缓存下一跳还在考虑
    struct tcb tcb; // 传输控制块
    dpdk_atomic32_t refcnt;
    // uint32_t hash; // 已经存储了头指针, 这个是否还需要
    uint32_t expire_time; // 过期时间是根据系统启动时间计算, 所以有好几十年
呢。
    void *recv_queue;
    void *send_queue;
    void *resend_queue; // 可能不需要
    struct vserver *vs;
    struct conn_hdr *hdr; // uint32_t position; // hash表中的position
struct conn_hdr;
    // 依据VS中是TCP代理还是HTTP/HTTPS/HTTP2代理, 确定该字段
    union {
        struct conn_body *proxy; // 代理方 TCP
        struct http *http; // HTTP
        struct https *https; // HTTPS
        struct http2 *http2; // HTTP2
    };
};

```

```
};
    struct list_head node;
} __attribute__((aligned(64)));
```

b、会话表管理

会话表的管理主要由DPDK哈希表(暂设置单线程哈希表容量500W)和滚动数组完成

结构体如下：

```
struct conn_manage {
    struct dpdk_hash *hash;
    uint16_t manual_seq;
    uint32_t roll_cur; // 就是当前时间，这里充当滚动数组索引 (roll_cur &
1023)
    uint32_t roll_next; // 保存上次清理的索引
    uint32_t roll_max; // 删除的最大跨度索引
    struct list_head roll_head[1024];
};
```

roll_head：数组中每个元素代表 **1 秒**。

当 **创建或刷新会话表** 时，需要将其加入或重新加入到数组中，以便按过期时间进行管理。

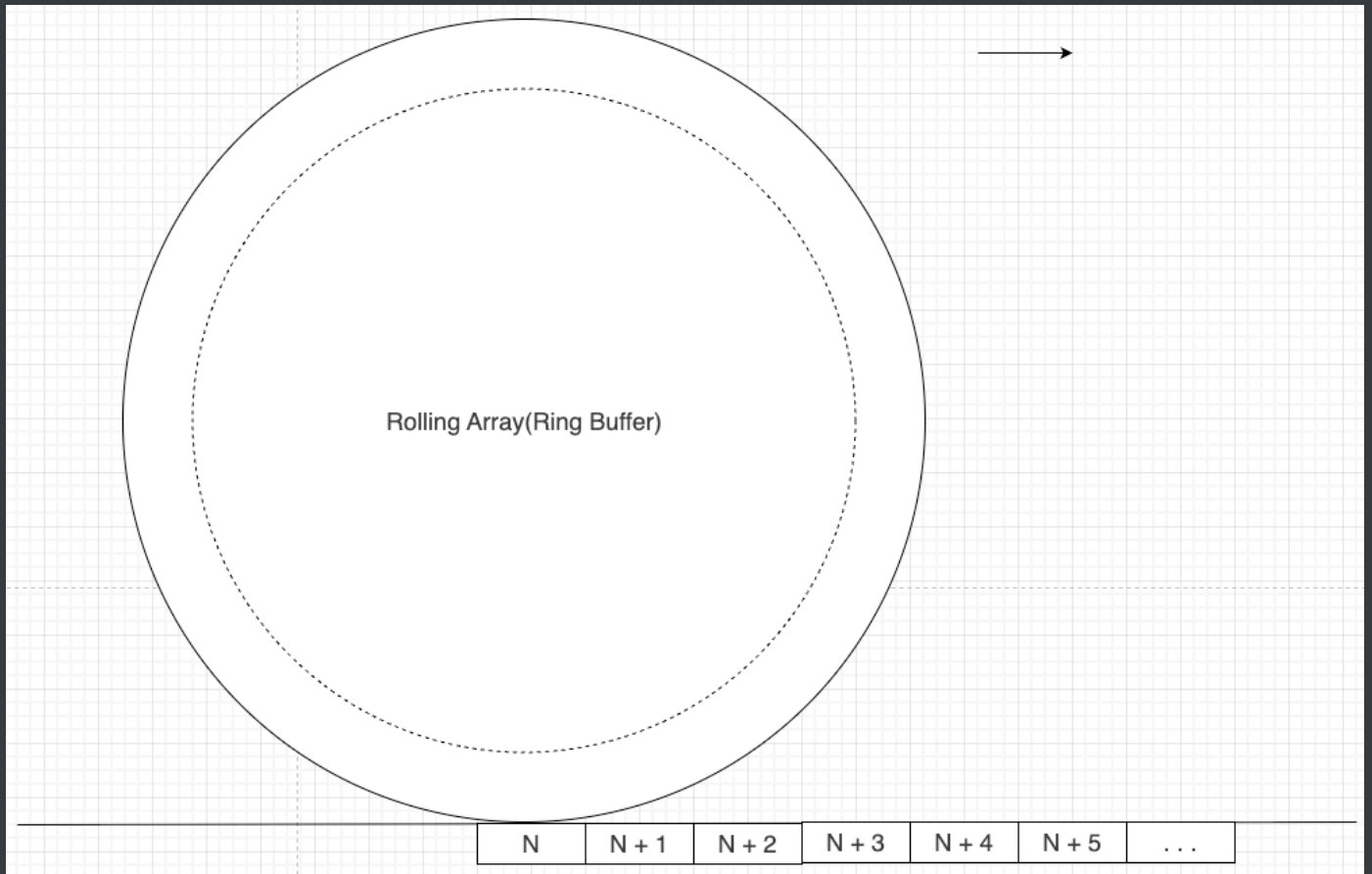
根据会话的 **过期时间 expired_time** 与线程本地的 **当前时间 cur_time**，计算挂载下标：

```
idx = ((expired_time - cur_time) + roll_cur) % 1024
```

然后将会话表挂载到：

```
roll_head[idx]
```

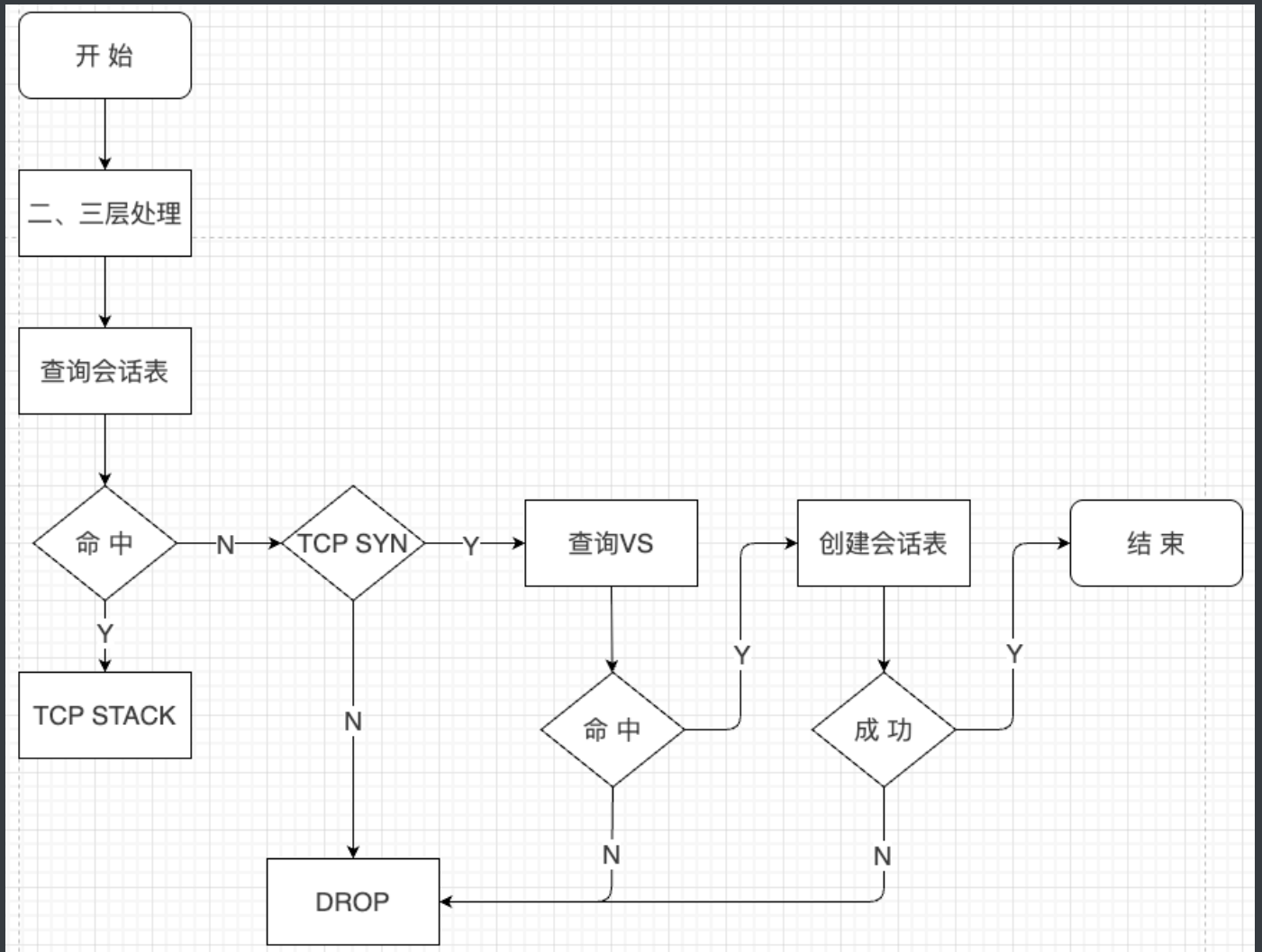
滚动数组示意图：



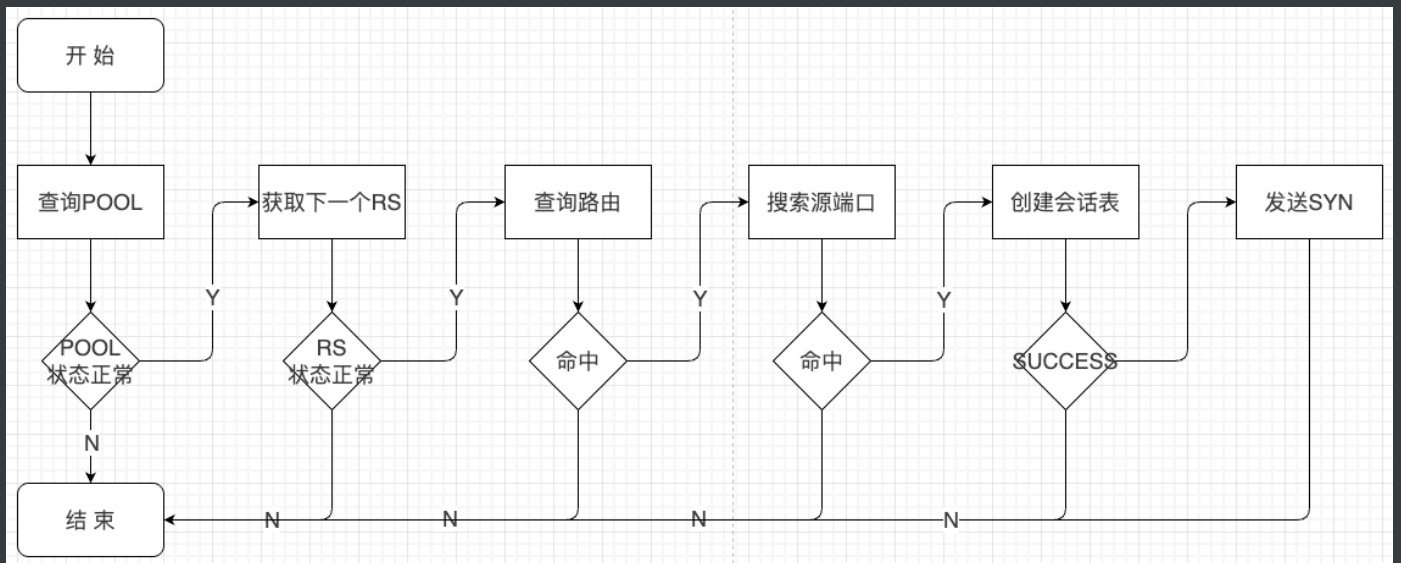
c、会话表创建

会话表创建流程如下：

客户端边：



服务端边：



d、会话表清理

- 自动清理

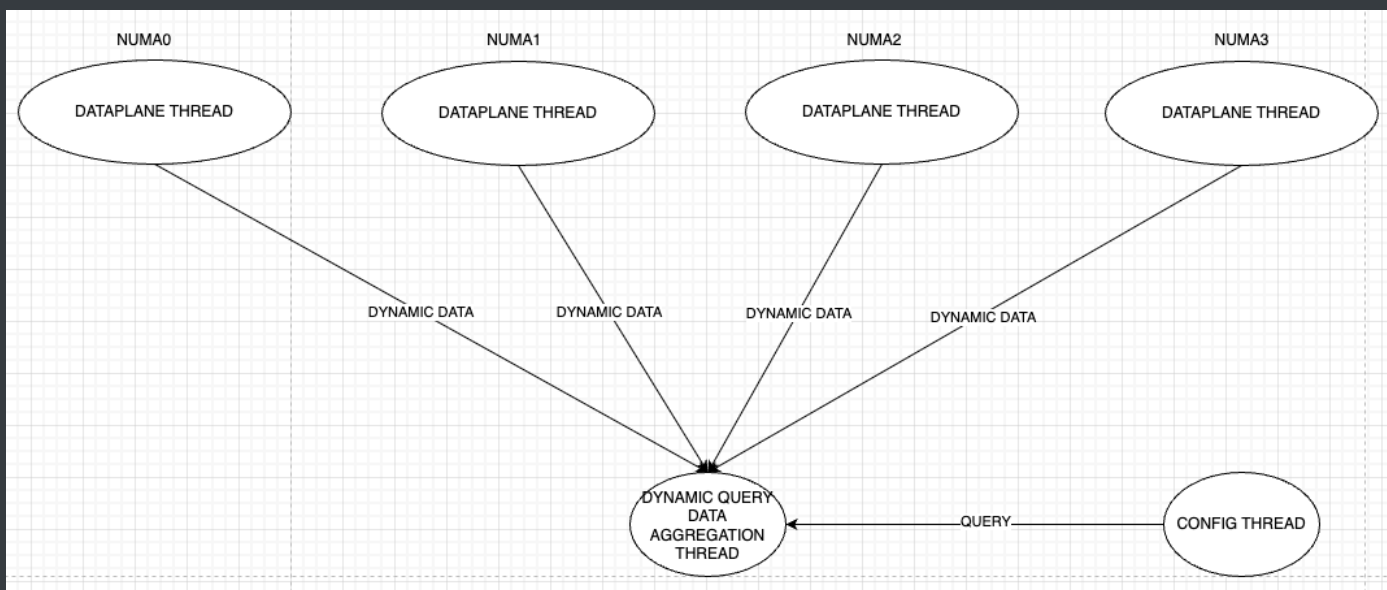
自动清理时比较roll_cur和roll_max

- 手动清理

当用户触发 **手动清理请求** 时：**配置线程** 向 **数据面线程** 和 **动态数据查询线程** 发送通知。所有会话的 **版本号统一加 1**。后续进行查询（无论是数据面查询还是用户查询）时，**版本号低于当前版本号** 的会话视为无效，不再命中。

e、会话表查询

DP中动态数据查询架构如下所示：



在设备中，每个会话表管理结构由数据面线程 (DP) 独立维护，采用无锁设计，仅供本线程访问，其他线程无法直接查询。让DP响应查询请求会影响DP的业务处理。为此，引入一个**集中式会话查询机制**：

- 新增一个**配置线程（动态数据查询线程）**，专门汇总并存储所有 DP 通知的信息，供外部查询使用，对性能要求不高；
- 当 DP 创建新会话时，会将会话表引用计数加 1，并通过**多写单读队列**发送通知给查询线程；
- 查询线程从队列中取出通知，复制所需的查询字段并存储（内部可采用有锁或无锁结构），随后将会话表引用计数减 1；
- 会话表的过期时间刷新时，若距离上次通知超过 3 秒，才再次上报，避免频繁更新；
- 会话握手、挥手等状态变更需要**实时通知**查询线程；

- 会话清理操作同样**实时通知**，可区分“清理单条”或“批量清理所有(手动清理)”(不能配置线程直接发信息让**动态数据查询线程**清理，因为每个DP线程是延迟清理，可能导致汇总的会话和实际DP线程的会话不一致)。

2、源端口搜索

源端口搜索分为端口搜索、端口回收以及端口集合的删除

a、端口搜索

- 核心思想：利用 NIC 的 RSS 哈希特性，对固定的 (sip, dip, dport)，每个具体的 sport 只会映射到唯一的接收线程（队列）。因此，每个线程只需维护属于自己的那一批 sport 候选集合，就可以在无锁前提下分配源端口，不会与其他线程冲突。
- 工作流：
 - 线程在第一次处理某个三元组 (sip, dip, dport) 时，建立/获取一个 port_thread。
 - 线程按批次扫描 1024..65535 的 sport，用预算的哈希快速判断该 sport 是否归本线程；若归本线程，则入队缓存。
 - 分配时从本线程缓存队列弹出一个 sport 返回（O(1)），无锁、无共享。
 - 当队列见底时，再按批次补充。
- 结构体

该结构体存放线程本地的port_hash表中，sip、dip和dport为key

```
struct port_thread {
    uint32_t sip;        // 源 IPv4 根据SNAT获取，每个不同的SIP、DIP以及
    // DPORT都会让SNAT引用计数
    uint32_t dip;        // 目的 IPv4
    uint16_t dport;      // 目的端口

    uint16_t max_port;   // 源端口扫描游标，范围 [1024, 65535]；用于批量
    // 补充端口时的下次起点
    uint32_t hash;       // 预计算的 (sip, dip, dport) 基础哈希，用于加速
    // (sip, sport, dip, dport) 的增量哈希

    struct port_queue {
```

```

        uint16_t cap;        // 环形队列最大容量（仅存“属于本线程”的
sport)
        uint16_t count;     // 累计分配给该三元组的 sport 数（含队列中与
已发放），便于观测/阈值控制
        uint32_t start;     // 环形队列头部索引（弹出位置）
        uint32_t end;       // 环形队列尾部索引（写入位置）
        // 仅保存“经 RSS 判定属于当前线程”的 sport，避免与其他线程重叠
        uint16_t id[(65536 - 1024) / cpu_count + N]; // N 为冗余容
量（如 1000），缓冲批量补充的波动
    } queue;
};

```

b、端口回收

当 **连接表** 回收时，会将端口归还给 **struct port_thread**。

需要注意：

- **TIME-WAIT** 状态下不会释放端口，因为会话表仍然存在，用于处理可能延迟到达的报文。
- 只有当连接从 **TIME-WAIT** 转换为 **CLOSED**（即定时器超时）时，才会回收所有相关资源并最终释放端口。

c、删除

当 **struct port_thread** 中所有端口均已被释放（即 $end - start == count$ ）时：

- 将该结构体加入定时器，延迟 **3 秒** 后再次检查。
- 如果在 3 秒后重新检查时， $end - start == count$ 仍然成立：
 - 释放该结构体。
 - 撤销对应的 **SNAT** 引用。
- 如果在 3 秒后检查时条件不再成立（ $end - start != count$ ）：
 - 不进行任何操作，保持结构体及 SNAT 引用不变。

3、延迟队列

在 **ARP/NDP 表** 中引入一个指向延迟队列的 **延迟指针**：

INCOMPLETE 状态处理：

当上层有数据需要发送，但 ARP/NDP 表中尚无对应 IP → MAC 的条目时：

- 创建一个 **INCOMPLETE** 状态的 ARP/NDP 表项。
- 让该表项的 **延迟指针** 指向一个 **延迟队列**。

延迟队列管理

- 每个延迟队列容量暂设为 **64 条**。
- 队列满时采用 **覆盖最早条目** 的策略。

状态变更：INCOMPLETE → COMPLETE

当 ARP/NDP 查询完成并更新为 **COMPLETE** 状态：

- 若存在缓存数据，则立即加入 **发送队列** 进行转发。

超时处理

如果 ARP/NDP 查询超时且未收到响应：

- 删除 **INCOMPLETE** 表项的同时，
- 也需从延迟队列中删除与该 **延迟指针** 关联的缓存数据（mbuf），避免无效占用内存。

4、(缓存下一跳)

由于 **配置、网络拓扑或动态路由** 等因素，**下一跳** 在连接的整个生命周期内都有可能发生变化。

因此，当前阶段是否对下一跳、MAC 地址及出接口进行缓存；后续在实现和优化时，再视实际需求决定是否引入这些缓存机制。